

Object-Oriented Programming (OOP)

Հարցազրույցի / քննության ամբողջական ուղեցույց

Այս presentation-ը ընդգրկում է OOP-ի բոլոր հիմնական թեմաները, որ հարցնում են interview-ում՝ 4 սյուները (Encapsulation, Inheritance, Polymorphism, Abstraction), inheritance-ի տեսակները, access modifier-ները, abstract class vs interface, object-ների միջև հարաբերությունները (association/aggregation/composition), SOLID սկզբունքները, և պատրաստի interview պատասխանները: Օրինակները Java-ով են, C++ զուգահեռներով:

0. Ի՞նչ է OOP-ը

OOP-ը (Object-Oriented Programming) ծրագրավորման մոտեցում է, որտեղ ծրագիրը կառուցվում է **Object-ների (օբյեկտների)** շուրջ, ոչ թե միայն ֆունկցիաների ու տրամաբանության:

Class -> blueprint / template (նախագիծ)
Object -> class-ի instance (իրական օրինակ հիշողության մեջ)

```
class Car {  
    String brand;  
}  
  
Car car1 = new Car(); // car1-ը Car class-ի object է
```

Անալոգիա. **Class** -ը տան նախագիծն է (գծագիր), **Object** -ը` այդ նախագծով կառուցված իրական տունը: Մեկ նախագծից կարելի է շատ տուն կառուցել:

1. Encapsulation (Կապսուլացում)

Տվյալների և մեթոդների միավորումն է մեկ class-ի մեջ, և տվյալների ուղիղ հասանելիության սահմանափակումը: Դաշտերը հայտարարում ենք `private`, հասանելիություն տալիս `getter/setter`-ներով:

```
class Person {  
    private String name;  
  
    public String getName()      { return name; }  
    public void setName(String n) { this.name = n; }  
}
```

Ինչու է կարևոր — 4 առավելություն

Data Hiding. Առանց encapsulation-ի ոչ ոք չի պաշտպանում տվյալները.

```
class BankAccount { public double balance; }  
acc.balance = -100000;           // սխալ արժեք, ոչ ոք չի արգելում
```

Encapsulation-ով վերահսկում ես.

```
class BankAccount {  
    private double balance;  
    public void setBalance(double b) {  
        if (b >= 0) this.balance = b;    // validation  
    }  
}
```

Security. `private String password;` նշանակում է, որ արտաքին class-երը չեն կարող ուղիղ դիպչել password-ին:

Better Control. Դու ես որոշում, թե ինչպես են տվյալները փոփոխվում.

```
public void setAge(int age) {  
    if (age > 0 && age < 120) this.age = age;    // setAge(-50) չի ընդունվի  
}
```

Easy Maintenance. Եթե ներքին ներկայացումը փոխես, բայց `getName()/setName()` -ը մնա նույնը, արտաքին կոդը չի կոտրվի:

Interview պատասխան

Encapsulation is the process of wrapping data and methods into a single unit (class) and restricting direct access to data using access modifiers.

C++ զուգահեռ. նույն գաղափարը `private` դաշտեր + `public` `getter/setter` մեթոդներ, plus `friend` -ը վերահսկված բացառություն է:

2. Inheritance (Ժառանգում)

Մեկ class-ը ստանում է մյուսի հատկություններն ու մեթոդները:

```
class Animal {  
    void eat() { System.out.println("Eating"); }  
}  
  
class Dog extends Animal { } // Dog-ը կարող է օգտագործել eat()
```

Առավելություններ

Code Reusability + Reduced Duplication. Ընդհանուր կոդը գրում ես մեկ անգամ base class-ում.

```
class Car { void start() {} }  
  
class BMW extends Car {}  
class Audi extends Car {}  
class Mercedes extends Car {} // բոլորը reuse են անում start()
```

Easier Maintenance. `Car.start()` -ը փոխելիս բոլոր child class-երը ավտոմատ ստանում են փոփոխությունը:

Inheritance-ի տեսակները

- | | | |
|-----------------|-------------------------|-------------------------|
| 1. Single | A -> B | (մեկ parent, մեկ child) |
| 2. Multilevel | A -> B -> C | (շղթա) |
| 3. Hierarchical | A -> B, A -> C | (մեկ parent, շատ child) |
| 4. Multiple | A, B -> C | (շատ parent, մեկ child) |
| 5. Hybrid | վերևիների համադրություն | |

Java-ում **Multiple Inheritance class-ներով չի աջակցվում** (Diamond Problem-ի պատճառով), բայց **interface-ներով հնարավոր է**.

```
interface A {}  
interface B {}  
class C implements A, B {} // OK
```

C++-ում, ի տարբերություն Java-ի, multiple inheritance class-ներով **թույլատրվում է** (`class C : public A, public B`), և Diamond Problem-ը լուծվում է `virtual` inheritance-ով:

3. Diamond Problem

Multiple inheritance-ի հիմնական խնդիրը.



Եթե **A**-ն ունի `show()`, և **B**, **C** երկուսն էլ ժառանգում են, ապա **D**-ն չգիտի որ `show()`-ն օգտագործի -> երկիմաստություն (ambiguity):

```
class A { void show() { System.out.println("A"); } }
class B { void show() { System.out.println("B"); } }
// class C extends A, B {} // Java-ում ARGELVAT E
```

Java-ի լուծումը — interface + override

Java 8-ից interface-ները կարող են ունենալ `default` մեթոդներ: Երկու նույնանուն default-ի դեպքում compile error է, պետք է override անել.

```
interface A { default void show() { System.out.println("A"); } }
interface B { default void show() { System.out.println("B"); } }

class C implements A, B {
    @Override
    public void show() {
        A.super.show(); // հստակ ընտրում ես որը
    }
}
```

C++-ի լուծումը — virtual inheritance

```
class A { public: void show() {} };
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {}; // A-ն մեկ անգամ է (virtual)
```


4. Polymorphism (Բազմաձևություն)

Poly (many) + Morph (forms). մեկ interface, տարբեր վարք:

Compile-time Polymorphism — Method Overloading

Նույն անուն, տարբեր parameters. compiler-ը ընտրում է ըստ argument-ների:

```
class MathUtil {
    int sum(int a, int b)          { return a + b; }
    int sum(int a, int b, int c) { return a + b + c; }
}
```

Runtime Polymorphism — Method Overriding

Child class-ը վերասահմանում է parent-ի մեթոդը. ընտրությունը runtime-ում է ըստ object-ի իրական տիպի:

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}

Animal a = new Dog();
a.sound();    // "Bark" -> runtime-ում որոշվում է
```

Overloading vs Overriding

	Overloading	Overriding
Ժամանակ	Compile-time	Runtime
Signature	տարբեր parameters	նույն signature
Inheritance	պետք չէ	պահանջվում է
Java keyword	-	@Override

Interview պատասխան

Polymorphism allows objects to take many forms. One interface can be used for different implementations.

C++ զուգահեռ. overloading = նույն; overriding-ը պահանջում է `virtual` մեթոդ base class-ում (Java-ում բոլոր non-static մեթոդները default-ով virtual են):

5. Abstraction (Աբստրակցիա)

Ցույց ենք տալիս միայն անհրաժեշտ տեղեկատվությունը, թաքցնում ենք ին implementation-ը:

```
abstract class Animal {  
    abstract void sound();    // ինչ, ոչ ինչպես  
}  
  
interface Vehicle {  
    void start();  
}
```

Անալոգիա. `car.start()` անելիս չգիտես, թե ներսում վառելիքի պոմպը ինչպես աշխատեց, մոմերը ինչպես կայծ տվեցին: Քեզ միայն `start()` -ն է հետաքրքրում: Դա abstraction-ն է:

Առավելություններ

Security -> օգտվողը իմանում է ինչ, ոչ ինչպես (թաքցված implementation)
Less Complexity -> պարզ interface բարդ ներսի փոխարեն
Better Design -> ընդհանուր contract, մաքուր կառուցվածք
Easy Maintenance -> նոր implementation ավելացնելը չի կոտրում մնացածը

Օրինակ՝ payment համակարգ.

```
interface Payment { void pay(double amount); }  
  
class CardPayment implements Payment { public void pay(double a) {} }  
class PayPalPayment implements Payment { public void pay(double a) {} }  
class CryptoPayment implements Payment { public void pay(double a) {} } // նոր, մնացածը չի փոխվում
```

Interview պատասխան

Abstraction is hiding implementation details and showing only essential information.

6. Abstract Class vs Interface

Ամենահաճախ հարցվող համեմատություններից մեկը:

```
abstract class Animal {  
    void eat() { System.out.println("Eating"); } // implementation ունի  
    abstract void sound(); // չունի  
}  
  
interface Vehicle {  
    void start(); // միայն contract  
}
```

	Abstract Class	Interface
Implementation	կարող է ունենալ	(Java 8+) միայն default/static
State (fields)	կարող է ունենալ	միայն public static final (constant)
Constructor	ունի	չունի
Multiple inheritance	ոչ (մեկ class)	այո (շատ interface)
Կիրառում	«is-a» + ընդհանուր կող	մաքուր contract / capability
Keyword	extends	implements

Կանոն. օգտագործիր **abstract class**, երբ ունես ընդհանուր կող/state child-երի համար;
օգտագործիր **interface**, երբ ուզում ես contract սահմանել կամ multiple inheritance-ի կարիք ունես:

7. Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Package
private	այո	ոչ	ոչ	ոչ
default	այո	այո	ոչ	ոչ
protected	այո	այո	այո	ոչ
public	այո	այո	այո	այո

Inheritance-ի ժամանակ ինչպես են աշխատում

```
class Parent {
    private int a = 10;
    protected int b = 20;
    public int c = 30;
}

class Child extends Parent {
    void print() {
        // a -> ERROR (private, child չի տեսնում)
        System.out.println(b); // OK (protected)
        System.out.println(c); // OK (public)
    }
}
```

Private-ը ժառանգվում է

Տեխնիկապես `private` դաշտը object-ի հիշողության մեջ կա, բայց child class-ը ուղիղ չի կարող օգտագործել: Հասանելի է միայն `public` getter-ով:

```
class Parent {
    private int age = 10;
    public int getAge() { return age; }
}

class Child extends Parent {
    void print() { System.out.println(getAge()); } // OK getter-ով
}
```

C++ զուգահեռ. `private` / `protected` / `public` նույն իմաստն ունեն, plus C++-ը ունի **inheritance-ի access** (`class B : private A`), որ Java-ում չկա:

8. Object-ների միջև հարաբերությունները

Inheritance-ը «is-a» է (Dog **is an** Animal): Բայց կա նաև «has-a» (composition/aggregation), որ interview-ում հաճախ մոռանում են:

Association (կապ)

Երկու անկախ object իրար հետ կապված են (օր. Teacher և Student):

Aggregation (has-a, թույլ)

Ամբողջը պարունակում է մասը, բայց մասը կարող է ապրել առանց ամբողջի.

```
class Department {  
    List<Teacher> teachers;    // Teacher-ը կարող է գոյություն ունենալ առանց Department-ի  
}
```

Composition (has-a, ուժեղ)

Մասը չի կարող ապրել առանց ամբողջի (ամբողջը ոչնչանալիս մասն էլ).

```
class House {  
    private Room room = new Room();    // Room-ը գոյություն չունի առանց House-ի  
}
```

Կանոն. **Composition over Inheritance** — հաճախ ավելի ճկուն է object-ներ կցելը (has-a), քան խորը inheritance շղթա կառուցելը (is-a):

9. SOLID սկզբունքները

Mid-level interview-ի համար կարևոր: Լավ OOP դիզայնի 5 սկզբունք.

S - Single Responsibility -> ամեն class մեկ պատասխանատվություն
O - Open/Closed -> բաց ընդլայնման, փակ փոփոխման
L - Liskov Substitution -> child-ը պետք է փոխարինի parent-ին առանց կոտրելու
I - Interface Segregation -> շատ փոքր interface > մեկ մեծ
D - Dependency Inversion -> կախված եղիր abstraction-ից, ոչ concrete class-ից

Single Responsibility. `Invoice` class-ը չպետք է և հաշվարկի, և տպի, և PDF սարքի. բաժանիր:

Open/Closed. Նոր payment տեսակ ավելացնելիս նոր class ես ավելացնում (`implements Payment`), հինը չես փոխում:

Liskov. Եթե `Bird` -ը ունի `fly()` , ապա `Penguin extends Bird` -ը կոտրում է Liskov-ը (penguin-ը չի թռչում) -> դիզայնը սխալ է:

Interface Segregation. Մի ստիպիր class-ին implement անել մեթոդներ, որ պետք չեն. բաժանիր interface-ը փոքրերի:

Dependency Inversion. Բարձր level կողը կախված լինի `Payment` interface-ից, ոչ `CardPayment` concrete class-ից:

10. Interview Short Answers

What is OOP?

A programming paradigm based on objects and classes.

4 pillars of OOP?

Encapsulation, Inheritance, Polymorphism, Abstraction.

Encapsulation?

Wrapping data and methods into a single unit and restricting direct access to data.

Inheritance?

Mechanism by which one class acquires properties and behaviors of another class.

Polymorphism?

Ability of an object to take multiple forms; one interface, many implementations.

Abstraction?

Hiding implementation details and showing only essential information.

Inheritance vs Encapsulation?

Inheritance = code reuse; Encapsulation = data protection.

Abstract class vs Interface?

Abstract class can have implementation and state; interface is a pure contract and enables multiple inheritance.

Overloading vs Overriding?

Overloading = same name, different parameters, compile-time; Overriding = same signature, runtime, needs inheritance.

Why no multiple inheritance in Java (classes)?

To avoid the Diamond Problem ambiguity; interfaces are used instead.

Composition vs Inheritance?

Inheritance = "is-a"; Composition = "has-a". Prefer composition for flexibility.

Cheat Sheet

4 PILLARS (մեկ նախադասությամբ):

Encapsulation -> Hide Data (private + getter/setter)
Inheritance -> Reuse Code (extends / is-a)
Polymorphism -> Many Forms (overload compile / override runtime)
Abstraction -> Hide Implementation (abstract class / interface)

INHERITANCE TYPES:

Single, Multilevel, Hierarchical, Multiple(interface), Hybrid

ACCESS (նեղից լայն):

private < default < protected < public

ABSTRACT CLASS vs INTERFACE:

abstract -> implementation + state, single inheritance
interface -> contract, multiple inheritance

RELATIONSHIPS:

is-a -> Inheritance
has-a -> Aggregation (թույլ) / Composition (ուժեղ)

SOLID: Single-Responsibility, Open/Closed, Liskov, Interface-Segregation, Dependency-Inversion

DIAMOND PROBLEM -> Java: interface + override | C++: virtual inheritance